

Reviewer Pack — Minimal p-Adic Logarithmic Capacity and SAT Clause Depth Ine...

Entry 48ee54c6e2c8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 15:33:10 UTC

Minimal p-Adic Logarithmic Capacity and SAT Clause Depth Inequality

Entry ID: 48ee54c6e2c8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 15:33:10 UTC

1. Conjecture

Field A (mathematical branch): p-Adic Number Theory (Logarithmic Capacity) **Field B** (complexity object): Boolean Satisfiability (Clause Depth)

Statement:

For any instance of a SAT problem with n clauses, the logarithmic capacity $C(n)$ of its p-adic valuation ring is strictly less than the clause depth $D(\varphi)$, i.e., $C(n) < \Theta(D(\varphi))$.

Rationale (proposer's reasoning):

Logarithmic capacity measures the size of the largest ball in a metric space that can be covered by a given set of points. This conjecture suggests that the structure encoded by p-adic valuation rings, which are used to encode logical formulas, is related to the complexity of proving their satisfiability, as measured by clause depth.

Taxonomy category: p-adic-log-capacity-sat-clause-depth (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9bff0b7675545d65

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all $n \leq 40$ SAT instances generated over 30 random seeds, the computed ratio $C(n)/D(\varphi) < 1.5$ AND the computation time for each instance is less than 240 seconds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - p-adic logarithmic capacity AND Boolean satisfiability - SAT clause depth INCLUSIVE p-adic number theory - logarithmic capacity p-adic valuation ring comparison with SAT clause depth

Top relevant hits considered: - [<http://arxiv.org/abs/math-ph/0512018v2>] On Phase Transitions for P -Adic Potts Model with Competing Interactions on a Cayley Tree - [<http://arxiv.org/abs/2408.00810v3>] p-adic Equiangular Lines and p-adic van Lint-Seidel Relative Bound - [<http://arxiv.org/abs/hep-th/9410058v3>] p-Adic description of Higgs mechanism I: p-Adic square root and p-adic light cone - [<http://arxiv.org/abs/1004.0653v2>] Exact Ramsey Theory: Green-Tao numbers and SAT - [<http://arxiv.org/abs/1908.01624v1>] Learned Clause Minimization in Parallel SAT Solvers - [<http://arxiv.org/abs/0707.3682v2>] On the p-adic Beilinson conjecture for number fields - [<http://arxiv.org/abs/1908.08424v1>] An introduction to p-adic period rings

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.6s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_sat_instance(n):
        variables = set(f'x{i}' for i in range(1, n+1))
        clauses = []
        for _ in range(n):
            clause = random.sample(variables, 2)
            clauses.append(clause)
        return clauses

    def tseitin_formula(clauses):
        literals = set()
        for clause in clauses:
            literals.update(clause)
        new_vars = {var: f'y{i}' for i, var in enumerate(literals, start=1)}
        new_clauses = []
        for literal in literals:
            new_clauses.append([new_vars[literal]])
        for clause in clauses:
            if literal not in clause:
                new_clause = [f'~{new_vars[literal]}']
                new_clause.extend(clause)
```

```

        new_clauses.append(new_clause)
    return new_clauses

def p_adic_valuation_ring_size(n):
    # Simplified approximation for demonstration
    return 2**n

def logarithmic_capacity(size):
    if size <= 0:
        return 0
    return math.log2(size)

def clause_depth(clauses):
    max_depth = 0
    for clause in clauses:
        depth = sum(1 for var in clause if var.startswith('~'))
        max_depth = max(max_depth, depth)
    return max_depth

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    clauses = generate_sat_instance(n)
    tseitin_clauses = tseitin_formula(clauses)
    size = p_adic_valuation_ring_size(n)
    cap = logarithmic_capacity(size)
    depth = clause_depth(tseitin_clauses)

    if cap >= 1.5 * depth:
        return {
            "metric_name": "C(n)/D( $\phi$ )",
            "metric_value": float('inf'),
            "instances_tested": len(n_values),
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "C(n) >= 1.5 * D( $\phi$ )"
        }

    results.append({
        "n": n,
        "cap": cap,
        "depth": depth
    })

mean_cap = sum(result["cap"] for result in results) / len(results)
mean_depth = sum(result["depth"] for result in results) / len(results)

return {
    "metric_name": "C(n)/D( $\phi$ )",
    "metric_value": mean_cap / mean_depth,
    "instances_tested": len(n_values),
    "n_max": max(result["n"] for result in results),
    "conjecture_holds": True,
    "counterexample": ""
}

```

```

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 3 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        for result in results:
            if not result["conjecture_holds"]:
                counterexample = result["counterexample"]
                first_failing_seed = seed
                break

        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bce449c6.py", line 103, in <module>
    trial_result = run_trial(seed)
    ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bce449c6.py", line 64, in run_trial
    clauses = generate_sat_instance(n)
    ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bce449c6.py", line 25, in generate_sat_instance
    clause = random.sample(variables, 2)
    ~~~~~^~~~~~
  File "/usr/lib/python3.12/random.py", line 413, in sample
    raise TypeError("Population must be a sequence. "
TypeError: Population must be a sequence. For dicts or sets, use sorted(d).

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete the required computations to verify the conjecture. | next: Investigate and fix the error in the test code that caused it to crash. Once fixed, re-run the test with a valid output to determine if the conjecture is supported or falsified.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14653
2	preregistration	ollama_remote	glm4:latest	0	0	9677
3	novelty	ollama_remote	glm4:latest	0	0	15598
4	novelty	ollama_remote	glm4:latest	0	0	10284
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17281
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9521
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9605
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	51168
9	judge	ollama_remote	glm4:latest	0	0	61677

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 199465 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/48ee54c6e2c8.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/48ee54c6e2c8.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/48ee54c6e2c8.tar.gz (if generated)